

COL106 / COL1101 — Programming with Data Structures

Practice Quiz Set (Unofficial) — 10 Practice Problems

Practice material generated for self-study, in the style of the CSE Moodle VPL problem set

Contents

How to use this practice set	2
Problem 1: The Fleet Console	3
Problem 2: The Data Pipeline Factory	6
Problem 3: Maximum Gain Window	8
Problem 4: The Growable Ring Buffer	10
Problem 5: Undo, Redo	12
Problem 6: The Interval Set	14
Problem 7: The Max Queue	16
Problem 8: The Bounded Buffer	18
Problem 9: The Sliding Window Maximum	20
Problem 10: The LRU Cache	22
Appendix: total marks summary	24

How to use this practice set

This is a self-study practice set, written in the same style, format and level of detail as the actual `student_code.py` / VPL-style lab assignments used in this course. It is **not** an official department document – it exists purely so you can rehearse solving one substantial data-structures problem, cold, in about two hours, under conditions that resemble a real programming quiz.

Each problem below corresponds to a folder in the accompanying archive:

```
Q<n>_<short-name>/
  student_code.py      <- write your solution here
  visible_cases/       <- sample input/expected-output pairs (2 per problem)
  hidden_tests.json    <- 20 additional test cases used only by grader.py
  grader.py            <- run this to grade your solution
```

To attempt a problem: open its `student_code.py`, read the problem description below, fill in every function/method marked `TODO` (keeping every given name, signature, and the I/O driver at the bottom of the file exactly as provided – the grader calls your code by name and feeds it input on `stdin` exactly the way the driver expects), and then run

```
python3 grader.py
```

from inside that problem’s folder. The grader will run your `student_code.py` against the 2 visible cases (shown in full, with a diff, if you get them wrong) and the 20 hidden cases (shown only as a pass/fail verdict – **AC** for a correct answer, **WA** for a wrong answer, **TLE** if your program did not finish inside the time limit, and **RE** if your program crashed), and finally print your total score out of the marks available for that problem.

The ten problems are ordered roughly by increasing difficulty and mirror the syllabus covered before this point in the course: OOP fundamentals, functional-style programming, asymptotic analysis and divide-and-conquer, arrays, linked lists, Set/Sequence/Vector ADTs, concurrency, and stacks/queues and their applications. Budget around **two hours per problem** if you are attempting one under quiz conditions – several of the later problems are intentionally long and unforgiving, exactly the way a real quiz-length problem would be.

A general piece of advice that applies to literally every problem here: get the **small, easy visible cases exactly right first**, by hand if necessary, before you worry at all about the large hidden performance tests. Every single problem in this set has at least a few hidden test cases specifically engineered so that a correct-but-slow (or correct-but-not-quite-worst-case) solution will time out. If your solution is right but slow, that is progress, not failure – go back and look at the *Expected Time Complexity* / performance guidance in that problem’s description for what you’re missing.

Problem 1: The Fleet Console

Topic: Object-Oriented Programming (classes, inheritance, encapsulation, dunder/special methods)
Difficulty: 1/5 (easiest in this set) **Suggested time:** 60–90 minutes **Total hidden marks:** 29 (across 20 hidden test cases; 2 additional visible cases are provided, ungraded, for your own debugging)

Story

A small car-rental company wants a bare-bones console to track its fleet of vehicles: what each vehicle earns, how many days it has spent rented out, and whether it is currently available or off the road for maintenance. You are asked to build this console as a small object-oriented program: a **Vehicle** base class capturing everything common to every vehicle in the fleet, two subclasses **Car** and **Truck** capturing how each kind of vehicle is actually priced, and a **FleetManager** class that owns every vehicle and answers every query the company might ask about the fleet as a whole. Along the way you will implement a handful of Python’s special (“dunder”) methods – `__str__`, `__eq__`, `__lt__`, `__gt__`, `__add__` and `__radd__` – and use them, rather than raw attribute access, to answer some of the trickier queries.

Pricing rules

Every vehicle has a unique string identifier and a base daily rental rate. A **Car** is priced simply: renting it for d days costs $\text{rate} * d$. A **Truck** additionally has a cargo capacity (in tons) and carries a surcharge of 5 per ton per day on top of its base rate, so renting a truck with cargo capacity c for d days costs $\text{rate} * d + 5 * c * d$. Every vehicle starts out with zero revenue earned, zero total days rented, zero completed rentals, and available (not under maintenance).

Commands

Your program reads a number of commands from standard input, one per line, and must respond to some of them by printing a single line of output (commands not listed as producing output produce none):

- **ADD CAR** `<id>` `<rate>` – register a new car with the given id and daily rate. If a vehicle with this id already exists, this command has no effect (it is not an error).
- **ADD TRUCK** `<id>` `<rate>` `<cargo>` – register a new truck with the given id, daily rate, and cargo capacity. Same “no effect if the id already exists” rule applies.
- **RENT** `<id>` `<days>` – rent out the vehicle with this id for the given number of whole days, immediately adding the appropriate cost to its running revenue total and adding `days` to its running total of days rented. If the id does not exist, **or the vehicle is currently under maintenance**, this command has no effect.
- **SERVICE** `<id>` **START** / **SERVICE** `<id>` **END** – mark the vehicle as under maintenance / available again. Calling either repeatedly is harmless (it simply leaves the flag in that state). Unknown ids have no effect.
- **REPORT** `<id>` – print the string form of the vehicle with this id (see below for the exact format), or the literal text **NOT FOUND** if no such vehicle exists.
- **COMPARE** `<id1>` `<id2>` – print the id of whichever of the two vehicles has **strictly** higher total revenue so far. If either id does not exist, print **NOT FOUND**. If the two vehicles have exactly equal revenue, print **TIE**. **You must determine this using the `<` and `>` operators**

on **Vehicle** objects (i.e. via `__lt__`/`__gt__`), not by comparing `.revenue` attributes directly in your `FleetManager` code.

- **FLEET_TOTAL** – print the combined revenue of every vehicle currently in the fleet, to exactly two decimal places. Compute this using Python’s built-in `sum()` over your vehicle objects (which only works once `__add__`/`__radd__` are implemented correctly) rather than writing your own manual accumulation loop over `.revenue` fields. Print 0.00 if the fleet is empty.
- **TOP** – print the id of the vehicle with the strictly highest revenue in the fleet (again, determined via comparisons between **Vehicle** objects, not raw numbers), breaking ties by the lexicographically smallest id among those tied. Print **NONE** if the fleet is empty.

Exact output format for REPORT

REPORT <id> must print exactly:

```
<id> [<TYPE>] revenue=<revenue> days=<days> status=<STATUS>
```

where <TYPE> is **CAR** or **TRUCK**, <revenue> is the vehicle’s total revenue formatted to exactly two decimal places, <days> is its total days rented (an integer), and <STATUS> is **AVAILABLE** or **MAINTENANCE**.

Worked example

```
ADD CAR C1 50
ADD TRUCK T1 80 2
RENT C1 3
RENT T1 2
REPORT C1
REPORT T1
```

C1 earns $50 * 3 = 150.00$ over 3 days. T1 earns $80 * 2 + 5 * 2 * 2 = 160 + 20 = 180.00$ over 2 days. The two **REPORT** lines therefore print:

```
C1 [CAR] revenue=150.00 days=3 status=AVAILABLE
T1 [TRUCK] revenue=180.00 days=2 status=AVAILABLE
```

What you must implement

In `student_code.py` you are given skeletons for **Vehicle**, **Car**, **Truck**, and **FleetManager**, along with a complete I/O driver at the bottom of the file that you must not modify (it parses the commands above and calls into your classes). Fill in every method marked **TODO**, keeping every given name and signature exactly as provided. You are free to add extra helper methods or attributes anywhere you like.

Notes on grading

The hidden test cases check: correct pricing for both vehicle types; that **ADD** on a duplicate id, and **RENT**/**SERVICE**/**REPORT**/**COMPARE** on an unknown id, all correctly have no effect (rather than crashing); that maintenance correctly blocks rentals; that **COMPARE** and **TOP** break ties correctly; that **FLEET_TOTAL** and **TOP** genuinely go through your operator overloads rather than reaching into `.revenue` directly (this is checked indirectly, by requiring `sum()` to work at all); and finally a couple of larger randomized scenarios (up to several thousand commands, and one stress test with several

thousand vehicles queried repeatedly via `TOP`) purely to confirm your implementation doesn't do anything pathologically slow – a dictionary-keyed `FleetManager` with $O(1)$ lookups per command comfortably clears every time limit here, so this problem is really about getting the object model and the dunder-method plumbing exactly right, not about algorithmic cleverness.

Problem 2: The Data Pipeline Factory

Topic: Functional-style programming (higher-order functions, composition, currying, memoization)

Difficulty: Difficulty 2/5 **Suggested time:** 75–100 minutes **Total hidden marks:** 37 (across 20 hidden test cases; 2 additional visible cases provided for debugging)

Story

A data-processing library wants its pipelines described *declaratively* – as a sequence of small, independent transformation stages – rather than as one big hand-written function that does everything at once. This is the essence of functional-style programming: instead of writing a monolithic procedure, you build small, reusable, higher-order functions (`compose`, `curry2`, `memoize`) once, and then use them repeatedly to assemble more complicated behaviour out of simple pieces. In this problem you will implement exactly these three pieces of generic machinery, and then build the rest of the program strictly on top of them.

Part A: the functional toolkit

You must implement three functions:

- `compose(*funcs)` returns a new function `c` such that `c(x) == funcs[0](funcs[1](...funcs[-1](x)...))` – i.e. the *last* argument is applied *first*, exactly the mathematical convention for function composition $f \circ g \circ h$. `compose()` with no arguments at all must return the identity function.
- `curry2(f)`, given a two-argument function `f(a, b)`, returns a curried version: `curry2(f)(a)(b) == f(a, b)` for every `a, b`.
- `memoize(f)` returns a wrapped version of `f` that caches results by argument tuple, so that calling the wrapped function twice with the same arguments only ever calls `f` itself once. Crucially, this must keep working correctly even when `f` is *recursive* and its recursive calls go through the *wrapped* version of itself – this is what lets memoization turn an exponential-time naive recursion into a linear-time one, and it is exactly what several of the hidden test cases below are designed to check.

Part B: a data pipeline built from map/filter stages

A pipeline is a sequence of stages, each either a “map” stage or a “filter” stage, described by one of the following op-codes:

- `MAP_ADD k` – replace every value `x` with `x + k`.
- `MAP_MUL k` – replace every value `x` with `x * k`.
- `FILTER_GT k` – keep only values `x` with `x > k`.
- `FILTER_LT k` – keep only values `x` with `x < k`.
- `FILTER_EVEN` – keep only even values.
- `FILTER_ODD` – keep only odd values.

You must build each stage’s actual one-argument function using `curry2` applied to a two-argument lambda (e.g. build the `MAP_ADD k` stage by partially applying a generic `lambda a, x: x + a` with `curry2`, rather than writing `lambda x: x + k` directly), and you must run the pipeline itself using Python’s built-in `map()` and `filter()` functions applied one stage after another (not manual loops that reimplement what `map/filter` already do).

Part C: a composed transform function

Separately from the pipeline, the input also specifies a list of named “transform” ops, each one of ADD1, DOUBLE, NEGATE, SQUARE, HALF (integer division by 2), given in the order they must be *applied* to a value (the first op in the list is applied first, the last op is applied last). You must build a single function out of this list using exactly one call to `compose()` – remember that `compose` applies its *last* argument first, so you will need to reverse the order of the looked-up functions before passing them to `compose` in order to get “first op in the list applied first” out the other end. If the list is empty, this function is simply the identity.

Part D: a memoized recursive Fibonacci

You must implement `fib(n)` – `fib(0) = 0`, `fib(1) = 1`, `fib(n) = fib(n-1) + fib(n-2)`, results reported modulo 1000000007 – as a *recursive* function decorated with your own `memoize`. A naive (non-memoized) recursive Fibonacci is exponential in `n`; several hidden tests call FIB with `n` in the hundreds, and/or call FIB with the same (moderately large) `n` many times in a row, specifically to time out a solution that isn’t genuinely memoized.

Input format and queries

```
N
<N space-separated integers -- the initial value list; blank line if N=0>
T
<T lines, each one pipeline stage op-code as above>
C
<C lines, each one transform op-code as above>
Q
<Q lines, each one of:>
  RUN                run the pipeline over the initial value list and print
                      the resulting list space-separated, or EMPTY if it is empty
  FIB n              print fib(n) mod 1000000007
  COMPOSE_EVAL x     print the composed transform function applied to x
```

Worked example

With initial values [1, 2, 3, 4, 5, 6, 7, 8], pipeline stages MAP_MUL 2 then FILTER_GT 5, and transform ops ADD1 then DOUBLE (applied in that order, i.e. `x -> (x+1)*2`):

```
RUN                ->  6 8 10 12 14 16    (after doubling: 2 4 6 8 10 12 14 16; keep >5)
FIB 10             ->  55
COMPOSE_EVAL 5     ->  12                  (5 -> 6 -> 12)
```

What you must implement

Fill in every TODO in `student_code.py`: `compose`, `curry2`, `memoize`, `make_stage`, `run_pipeline`, `build_transform`, and `fib`. The I/O driver at the bottom is complete and must not be modified. Re-implementing the caching logic of `memoize` by hand *inside* `fib` instead of applying the `memoize` decorator to it defeats the purpose of this exercise and will not receive credit even where the output happens to be correct.

Problem 3: Maximum Gain Window

Topic: Asymptotic analysis and Divide-and-Conquer **Difficulty:** Difficulty 3/5 **Suggested time:** 90–120 minutes **Total hidden marks:** 44 (across 20 hidden test cases; 2 additional visible cases provided for debugging)

Story

A trader has a list of n daily profit/loss values `arr[0]`, `arr[1]`, ..., `arr[n-1]` (each may be positive, negative, or zero). She wants to know the largest total gain achievable by holding a position over some *contiguous* run of days `arr[l]`, `arr[l+1]`, ..., `arr[r]` ($l \leq r$; a single day, i.e. $l == r$, is always an allowed “window”). This is the classical **Maximum Subarray Sum** problem, and this problem asks you to implement it *two* different ways, so that you can see for yourself, on your own machine, exactly why one of the two scales to large inputs and the other simply cannot.

Method 1: Brute force, $\Theta(n^2)$

Try every pair of indices (i, j) with $i \leq j$ and compute the sum `arr[i] + ... + arr[j]` directly for each, keeping the best sum seen. This is the algorithm you would write in your very first week of programming: obviously correct, and obviously useless once n reaches the tens of thousands.

Method 2: Divide and conquer, $\Theta(n \log n)$

Split the range `[low, high]` at `mid = (low + high) // 2`. The best window either lies *entirely* within the left half `[low, mid]`, lies *entirely* within the right half `[mid+1, high]`, or *straddles* the midpoint (i.e. it contains both `arr[mid]` and `arr[mid+1]`). Recursively solve the first two cases. For the third case you cannot recurse – you must scan outward from the midpoint in both directions, maintaining a running sum, to find the best “crossing” window in $\Theta(\text{high} - \text{low} + 1)$ time. It is essential that this crossing computation is genuinely linear (i.e. that you do **not** recompute the running sum from scratch for every candidate endpoint) – that is precisely what makes the overall recurrence $T(n) = 2T(n/2) + \Theta(n)$ work out to $\Theta(n \log n)$ rather than something worse, and it is exactly what several hidden performance tests below are designed to catch if you get it wrong.

What you must implement

Three functions in `student_code.py`, with exactly the given signatures:

- `max_crossing_sum(arr, low, mid, high)` – returns the maximum sum of a window `arr[i..j]` with $\text{low} \leq i \leq \text{mid}$ and $\text{mid} < j \leq \text{high}$ (i.e. a window that includes both `arr[mid]` and `arr[mid+1]`, and stays within `[low, high]`), in $\Theta(\text{high} - \text{low} + 1)$ time.
- `max_subarray_dc(arr, low, high)` – returns the maximum window sum of `arr[low..high]` using the divide-and-conquer method above, calling `max_crossing_sum` for the crossing case.
- `max_subarray_brute(arr, low, high)` – returns the maximum window sum of `arr[low..high]` using the naive $\Theta((\text{high} - \text{low} + 1)^2)$ double loop. Do **not** call `max_subarray_dc` from inside this function, and do not try to make it fast – that is not the point of this function; it exists purely as an independently-trustworthy reference you (and the grader) can cross-check the divide-and-conquer answer against on inputs small enough to afford it.

Input format and queries

```
n
<n space-separated integers -- arr[0..n-1]>
q
<q lines, each one of:>
  BRUTE          report max_subarray_brute over the WHOLE array
  DC             report max_subarray_dc over the WHOLE array
  RANGE l r      report max_subarray_dc over arr[l..r] (0-indexed, inclusive)
```

Worked example

```
arr = [-2, 1, -3, 4, -1, 2, 1, -5, 4]
BRUTE  -> 6      (the window [4, -1, 2, 1], i.e. arr[3..6])
DC      -> 6
RANGE 3 6 -> 6
```

Notes on grading

Small correctness cases (single element, all-negative, all-positive, windows that straddle the midpoint exactly, zeros, negative numbers, and several randomized small arrays with both **BRUTE** and **DC** cross-checked against each other) are worth the bulk of the “correctness” marks. Separately, several hidden test cases run **DC/RANGE only** (never **BRUTE**, since even a *correct* $\Theta(n^2)$ algorithm would be far too slow) on arrays of up to 150,000 elements, including adversarial patterns (all-negative, strictly increasing, and repeated **RANGE** queries over arbitrary sub-windows of a large array) – these exist specifically to confirm that your `max_subarray_dc` is genuinely $\Theta(n \log n)$ and not, say, a disguised $\Theta(n^2)$ algorithm that merely happens to produce correct answers on the small cases. If you find yourself unsure whether your divide-and-conquer solution is *actually* dividing and conquering, or is secretly falling back on brute force somewhere, this is precisely the kind of bug the large hidden cases will surface as a **TLE** rather than a **WA** – pay close attention to which verdict you get.

Problem 4: The Growable Ring Buffer

Topic: Arrays and Amortized Analysis **Difficulty:** Difficulty 3/5 **Suggested time:** 90–110 minutes **Total hidden marks:** 54 (across 20 hidden test cases; 2 additional visible cases provided for debugging)

Story

Python’s `list` already gives you a dynamic array for free, which makes it tempting to build a queue on top of it with `list.append(x)` for enqueue and `list.pop(0)` for dequeue. **Do not do this here.** `list.pop(0)` is $\Theta(n)$, because removing the first element of a Python list requires shifting every remaining element one slot to the left; over n dequeues that is $\Theta(n^2)$ in total – exactly the trap this problem exists to catch.

Instead, you will build a `DynamicCircularQueue` directly on top of a *fixed-size* Python list used as a raw array (a “ring buffer”): a contiguous block of slots, a `head` index that wraps around using modular arithmetic, and a `count` of how many elements are currently stored. Enqueuing writes into the slot just past the current tail; dequeuing simply advances the head index and decrements the count – no element is ever shifted, in either operation.

The resize policy (must be followed exactly)

A ring buffer has a fixed capacity, and yours must grow and shrink automatically so that operations remain **amortized $O(1)$** while memory usage stays proportional to the number of elements actually stored. You must follow this *exact* policy, since several hidden test cases query the capacity directly and expect it to match precisely:

- The queue starts with capacity 2.
- Immediately before an enqueue that would make `count` exceed the current `capacity` (i.e. `count == capacity` right before this enqueue), you must **double** the capacity (`capacity = capacity * 2`) before placing the new element.
- Immediately after a dequeue, if `capacity > 2` **and** `count <= capacity // 4` (integer division), you must **halve** the capacity (`capacity = capacity // 2`). The capacity must never be allowed to drop below 2.
- A resize (grow or shrink) must copy every currently-stored element, in queue order (current front first), into a freshly allocated list of the new size, resetting the head index of the new buffer to 0.

This “double when full, halve when a quarter full” policy is the standard way to achieve amortized $O(1)$ enqueue/dequeue while guaranteeing the buffer is never more than a small constant factor larger than it needs to be – but only if you get the exact thresholds right. A queue that shrinks every time it drops below *half* full, for instance, can be made to thrash: alternating single enqueues and dequeues right at a capacity boundary can force a resize on every single call, destroying the amortized bound entirely even though every individual answer is still correct. Several hidden test cases are specifically built around this exact failure mode.

Commands

q
<q lines, each one of:>

```
ENQUEUE x
DEQUEUE
FRONT
SIZE
CAPACITY
```

DEQUEUE and FRONT on an empty queue must print **EMPTY** rather than raising an error.

Worked example

```
ENQUEUE 1
ENQUEUE 2
ENQUEUE 3          (capacity doubles from 2 to 4 just before this enqueue)
ENQUEUE 4
ENQUEUE 5          (capacity doubles from 4 to 8 just before this enqueue)
CAPACITY           -> 8
DEQUEUE DEQUEUE DEQUEUE DEQUEUE    (count drops to 1; 1 <= 8//4=2, so capacity halves to 4)
CAPACITY           -> 4
```

What you must implement

Complete the `DynamicCircularQueue` class in `student_code.py`: `__init__`, `_resize`, `enqueue`, `dequeue`, `front`, `__len__`, and `is_empty`. The I/O driver at the bottom (which reads `dq.capacity` directly for the **CAPACITY** command, so make sure you store it under exactly that attribute name) is complete and must not be modified.

Notes on grading

Small correctness cases check FIFO order, wraparound reuse of slots after a dequeue-then-enqueue, the exact doubling sequence, the exact halving sequence, and that capacity never drops below 2. Larger hidden cases run hundreds of thousands of enqueue/dequeue operations – including a pure-enqueue stress test, a random mixed-operation stress test, and (most unforgivingly) an alternating single-enqueue/single-dequeue stress test performed 50,000 times in a row right at a capacity boundary – all under a tight time limit that a genuinely amortized $O(1)$ implementation clears comfortably, and that a subtly-wrong resize policy (even one that is still functionally correct!) will not.

Problem 5: Undo, Redo

Topic: Linked Lists / Stacks and their Applications **Difficulty:** Difficulty 3/5 **Suggested time:** 75–100 minutes **Total hidden marks:** 45 (across 20 hidden test cases; 2 additional visible cases provided for debugging)

Story

You are building the undo/redo engine of a bare-bones text editor. The editor holds a single string, `document`, which starts out empty. Every edit either appends text to the end of the document (`TYPE`) or removes some number of characters from its end (`DELETE`). At any point the user may `UNDO` the most recent not-yet-undone edit, or `REDO` the most recently undone edit – following the usual rule you already know from every text editor you have ever used: making a brand new edit after undoing something throws away the ability to redo whatever you had undone (the “future” you rewound out of is gone the moment you type something new).

Commands

<code>TYPE <text></code>	Append <code><text></code> to the document. Remember the document's <code>PREVIOUS</code> contents on an undo history, and throw away any redo history.
<code>DELETE k</code>	Remove the last <code>k</code> characters from the document (if <code>k</code> is at least the document's current length, it becomes empty). Exactly like <code>TYPE</code> , this remembers the previous contents and throws away the redo history.
<code>UNDO</code>	Revert the document to whatever it was immediately before the most recent not-yet-undone <code>TYPE</code> or <code>DELETE</code> . The state being left (i.e. the document just before this <code>UNDO</code>) becomes redoable. No effect if there is nothing to undo.
<code>REDO</code>	Re-apply the most recently undone edit, restoring the document to what it was immediately before the matching <code>UNDO</code> . The state being left becomes undoable again. No effect if there is nothing to redo.
<code>PRINT</code>	Print the document's current contents, or the literal text <code><EMPTY></code> if the document is currently the empty string.

The required implementation: two stacks, each a genuine linked list

You must implement this with **two stacks** – an undo stack and a redo stack, each holding past versions of the whole document string – and, critically, you must implement the Stack itself as a genuine **singly linked list** (using the `Node-based LinkedStack` class provided in the skeleton), not with a Python `list`, `collections.deque`, or any other pre-built container. The entire point of this exercise is practising building and using your own linked-list-backed Stack ADT, and then seeing how two of them together give you undo/redo “for free” once the ADT itself is right.

Worked example

```
TYPE hello
TYPE _world
PRINT      -> hello_world
UNDO
PRINT      -> hello
REDO
PRINT      -> hello_world
```

A slightly trickier example showing the “new edit clears redo” rule:

```
TYPE abc
TYPE def
UNDO      (document is now "abc"; "abcdef" is now redoable)
TYPE xyz  (document is now "abcxyz"; the "abcdef" redo is now GONE)
REDO      (nothing to redo -- no effect)
PRINT     -> abcxyz
```

What you must implement

In `student_code.py`, complete the `_Node`-based `LinkedList` class (`push`, `pop`, `peek`, `is_empty`, `clear`, `__len__` – every one of these must run in $O(1)$; in particular, `clear()` must **not** pop elements off one at a time in a loop – simply detach the whole chain from the stack’s own state in $O(1)$ and let it be garbage collected), and then the `UndoRedoEditor` class built on top of two `LinkedList` instances (`type_text`, `delete_last`, `undo`, `redo`, `current`). The I/O driver at the bottom is complete and must not be modified.

Notes on grading

Hidden test cases check: basic type/delete/print; that undo restores exactly the previous document contents (not, say, a re-computed or approximate version); that redo restores exactly the state that was undone; that a brand-new edit after an undo correctly discards the redo history (a very common bug is to forget this, or to only *partially* clear it); that undo/redo on an empty history are silent no-ops rather than crashes; deep chains of undo that unwind all the way back to the original empty document; and a performance case with several thousand interleaved operations to confirm your `LinkedList` operations are genuinely $O(1)$ each rather than, say, secretly walking the whole chain on every push or pop.

Problem 6: The Interval Set

Topic: Set ADT, implemented on a Sorted Doubly Linked List **Difficulty:** 4/5 **Suggested time:** 110–130 minutes **Total hidden marks:** 58 (across 20 hidden test cases; 2 additional visible cases provided for debugging)

Story

A set of integers can be enormous – imagine “every integer from 0 to one billion except a handful of gaps” – far too large to store one element at a time. But if the set happens to consist of a small number of contiguous *ranges* of integers, it can be represented far more compactly: as a collection of disjoint intervals. This problem asks you to build exactly such a representation, an `IntervalSet`, supporting adding a range of integers to the set, removing a range of integers from the set, and testing membership – all backed by a **sorted doubly linked list of interval nodes** (not a Python `list`, not a Python `set`, not any other built-in container to hold the intervals themselves).

Representation

Represent each interval as the half-open range $[lo, hi)$ – containing every integer $lo, lo+1, \dots, hi-1$ but **not** hi itself (the same convention as Python’s `range()` and slicing, which is exactly what makes “adjacent” ranges like $[1,5)$ and $[5,10)$ unambiguously mergeable into $[1,10)$: together they cover $1..9$ with no gap and no overlap). At all times, the linked list must represent the current set with the *smallest possible* number of intervals: sorted in increasing order, each one non-empty, with a gap of at least 1 between the hi of one interval and the lo of the next. Two intervals that touch or overlap must always be merged into one before control returns to the caller – never leave the structure in a state where two neighbouring nodes could have been combined into one.

Operations

ADD $lo\ hi$	Add every integer in $[lo, hi)$ to the set. May need to merge with, or completely swallow, any number of existing intervals it overlaps or touches. No effect if $lo \geq hi$.
REMOVE $lo\ hi$	Remove every integer in $[lo, hi)$ from the set. May split an existing interval into two pieces, shrink an interval from one end, or delete some number of intervals entirely, all within the same call. No effect if $lo \geq hi$.
CONTAINS x	Report whether x is currently in the set (YES/NO).
COUNT_INTERVALS	Report how many intervals (linked-list nodes) the set currently consists of.
SIZE	Report the TOTAL number of integers currently in the set (the sum of $hi - lo$ over every interval).
DUMP	Print every interval, in increasing order, each formatted as <code>"[lo,hi)"</code> , space-separated -- or the literal word <code>EMPTY</code> if the set is currently empty.

Worked example (“bridging” ADD)

If the set currently holds the two disjoint intervals $[1,3)$ and $[5,7)$, then `ADD 0 20` must replace **both** of them (and everything in between) with the single interval $[0,20)$. Your `ADD` needs to walk however many existing intervals the new range touches or overlaps, merge them all together with the new range into one combined interval, remove every one of the swallowed nodes, and insert exactly one new node in their place. Symmetrically, `REMOVE 5 8` applied to a set containing $[0,10)$ must leave behind *two* intervals, $[0,5)$ and $[8,10)$ – the piece that survives on the left of the removed range, and the piece that survives on the right.

```
ADD 1 3
ADD 5 7
ADD 0 20
DUMP                -> [0,20)

ADD 0 10
REMOVE 4 6
DUMP                -> [0,4) [6,10)
CONTAINS 4          -> NO
CONTAINS 3          -> YES
```

What you must implement

Complete the `IntervalSet` class in `student_code.py`: the low-level doubly-linked-list helpers `_insert_after` and `_remove`, and the six public operations `add_range`, `remove_range`, `contains`, `count_intervals`, `total_size`, and `dump`. The I/O driver is complete and must not be modified.

Notes on grading

Small correctness cases check disjoint ranges staying separate, overlapping ranges merging, *adjacent* (touching but not overlapping) ranges merging, a single `ADD` bridging and swallowing several existing intervals at once, a single `REMOVE` splitting an interval, a single `REMOVE` eating across several existing intervals partway, removing a range with no intersection at all (no effect), clearing the entire set, adding a duplicate range twice (idempotent), negative coordinates, and invalid ranges (`lo >= hi`, no effect). A battery of randomized correctness tests then checks arbitrary interleavings of `ADD/REMOVE/CONTAINS` against ground truth. Finally, one hidden performance test issues 20,000 `ADD/REMOVE` calls over a coordinate space of size up to 10^9 – an implementation that does anything resembling “materialize the set as a literal collection of individual integers” will not come close to finishing in time; your implementation must only ever touch the (small number of) intervals actually relevant to each call.

Problem 7: The Max Queue

Topic: Queue ADT built from Stacks; amortized analysis **Difficulty:** 4/5 **Suggested time:** 100–120 minutes **Total hidden marks:** 55 (across 20 hidden test cases; 2 additional visible cases provided for debugging)

Story

A queue and a stack look like opposites – one is FIFO, the other LIFO – but you can build a perfectly good queue using nothing but two stacks, and it is a classic, instructive exercise in exactly the kind of amortized-cost reasoning this course cares about. On top of that queue, you will also support an **$O(1)$ -amortized MAX query** (“what is the largest element currently in the queue?”), using a second classic trick: a stack that tracks its own running maximum.

Part A: MaxStack

A **MaxStack** behaves exactly like an ordinary stack (**push**, **pop**, **peek**, **is_empty**, **__len__**), but additionally supports **peek_max()**: return the largest element currently anywhere in the stack, in $O(1)$ time. The standard trick is to keep a second, parallel stack of “running maximums”: every time you push an element onto the main stack, also push, onto the running-max stack, the larger of (that new element) and (whatever the running-max stack’s current top already is) – so the running-max stack’s top is always the maximum of everything below it in the main stack, and popping both stacks’ tops together keeps this invariant intact.

Part B: MaxQueue, built from two MaxStacks

Keep an **in_stack** (freshly enqueued elements, most recent on top) and an **out_stack** (elements ready to be dequeued, oldest on top). **ENQUEUE** always just pushes onto **in_stack** – $O(1)$, always. **DEQUEUE** and **FRONT** need the *oldest* element, buried at the bottom of **in_stack**, so whenever **out_stack** is currently **empty**, pop *every* element off **in_stack** and push it onto **out_stack** (this reverses their order, putting the oldest element on top of **out_stack** where it belongs). Do this “shift” only when **out_stack** is empty – never migrate elements back from **out_stack** to **in_stack**, and never shift when **out_stack** already has something on it. This exact condition is what makes the whole scheme amortized $O(1)$ per operation: any single **DEQUEUE** might occasionally move a large number of elements across in one go, but each element only ever makes that trip once across its entire lifetime in the queue. Finally, **GET_MAX** returns the larger of **in_stack.peek_max()** and **out_stack.peek_max()** (skipping whichever stack, if either, happens to be empty).

Why this is trickier than it looks

Getting the amortized analysis subtly wrong is easy to do without noticing on small tests: re-shifting on *every* **DEQUEUE** call regardless of whether **out_stack** is already non-empty still produces correct *output*, but is quietly $\Theta(n)$ per call instead of amortized $O(1)$. Several of the hidden test cases run on the order of a hundred thousand interleaved operations specifically to catch exactly that mistake, as well as a subtler one: making sure **GET_MAX** correctly tracks the maximum *across a shift* (i.e. that the maximum doesn’t get “lost” the moment the element holding it moves from **in_stack** to **out_stack**).

Commands

```
q
<q lines, each one of:>
  ENQUEUE x
  DEQUEUE
  FRONT
  MAX
  SIZE
```

DEQUEUE, FRONT, and MAX on an empty queue must print EMPTY.

Worked example

```
ENQUEUE 5
ENQUEUE 2
ENQUEUE 9
ENQUEUE 1
MAX                -> 9
DEQUEUE DEQUEUE DEQUEUE    (removes 5, 2, 9 in that order)
MAX                -> 1
```

What you must implement

Complete `MaxStack` (`push`, `pop`, `peek`, `peek_max`, `is_empty`, `__len__`) and `MaxQueue` (`enqueue`, `_shift`, `dequeue`, `front`, `get_max`, `is_empty`, `__len__`) in `student_code.py`. The I/O driver is complete and must not be modified.

Notes on grading

Small correctness cases check basic FIFO order, basic max tracking, that the max updates correctly once the element holding it is actually dequeued, ties in the maximum, negative numbers, and a deliberately constructed sequence that forces a “shift” to happen in the middle of the trace so you can check the max survives it correctly. Larger hidden cases include: 60,000 alternating single enqueue/dequeue pairs (which would be catastrophically slow under an always-reshift implementation), a 50,000-operation stress test with heavy interleaved `MAX` queries, and a 200,000-element pure-enqueue-then-pure-dequeue test – all under tight time limits designed around genuinely amortized $O(1)$ behaviour.

Problem 8: The Bounded Buffer

Topic: Concurrency (threads, locks, condition variables) **Difficulty:** 4/5 **Suggested time:** 90–120 minutes **Total hidden marks:** 63 (across 20 hidden test cases; 2 additional visible cases provided for debugging)

Story

You are implementing the classic **Producer-Consumer** synchronization problem: a fixed-capacity buffer shared between some number of producer threads and some number of consumer threads. A producer calls `put(item)` to add an item to the buffer; if the buffer is currently full, `put()` must **block** – the calling thread must sleep, consuming no CPU, rather than spin-checking in a loop – until a consumer makes room. A consumer calls `get()` to remove and return the oldest item in the buffer (FIFO order); if the buffer is currently empty, `get()` must block until a producer adds something.

What you must use

You must implement this using `threading.Lock` and `threading.Condition` from Python’s standard library – **not** `queue.Queue`, **not** `multiprocessing`, and **not** a manual busy-wait loop (e.g. `while full: time.sleep(0.001)`), which technically produces correct answers but wastes CPU and misses the point of the exercise entirely. A condition variable lets a thread atomically release its lock and go to sleep in one step, and be woken up efficiently by another thread the moment the condition it is waiting for might have become true – that mechanism is exactly what you are practising here.

Implement the class `BoundedBuffer`:

<code>__init__(self, capacity)</code>	Set up an empty buffer of the given fixed capacity.
<code>put(self, item)</code>	Block (without busy-waiting) until there is room, then insert item at the back, and wake up any consumer thread that might be waiting for a new item.
<code>get(self)</code>	Block (without busy-waiting) until the buffer is non-empty, then remove and return the item at the front, and wake up any producer thread that might be waiting for room to open up.

A standard, robust design uses **one lock** together with **two** `Condition` objects built on that same lock – conventionally named `not_full` and `not_empty` – so a thread blocked in `put()` is only ever woken by a `get()` (and vice versa), rather than every waiting thread being woken on every single change. You may instead use a single shared `Condition` if you prefer, provided your implementation is genuinely free of races and busy-waiting.

Important: whenever you wait on a condition, you must re-check the condition you were waiting for in a `while` loop after waking up (never an `if`-statement) – “spurious wakeups”, and the fact that several threads can be woken for the same event even though only one of them can actually proceed, are exactly why `while cond_not_yet_true: c.wait()` is correct and `if cond_not_yet_true: c.wait()` is not.

How correctness is checked (no flaky timing assumptions)

The rest of `student_code.py` (a complete driver you must not modify) spins up the requested number of producer and consumer threads against your `BoundedBuffer`. Every producer generates a distinct, uniquely tagged sequence of items; every consumer collects whatever items it manages to `get()`. Once every thread has finished, the driver itself checks that (a) every item that was produced was consumed *exactly once* – nothing lost, nothing duplicated, nothing left behind – and (b) the total count of consumed items matches the total count produced. It prints `OK` followed by the verified item count if both checks pass, or `FAIL` otherwise. Because the correctness check is about *what* was eventually produced and consumed rather than *when*, a correct, deadlock-free implementation will reliably print the same `OK` / count every single time it is run, with no flakiness – if your program instead prints `FAIL`, hangs until the grader’s timeout kills it, or crashes, your synchronization has a genuine bug (most commonly: notifying the wrong condition, checking a condition with `if` instead of `while`, or reading/writing the shared buffer somewhere without holding the lock).

Input format

```
capacity
number_of_producer_threads
items_per_producer_thread
number_of_consumer_threads
```

Notes on grading

Hidden test cases sweep across a wide range of configurations: single-producer/single-consumer, many-producers/one-consumer, one-producer/many-consumers, balanced many-to-many, a very large buffer (producers rarely block), and – the most demanding case – buffer capacity 1 (maximal contention, forcing blocking to actually work correctly rather than “usually happening to work” with a larger, more forgiving buffer). Several configurations are additionally repeated multiple times with different thread counts specifically to surface any residual race condition that might only manifest occasionally. The largest hidden tests push several hundred items through capacity-1 buffers across multiple producer/consumer threads, under a generous but finite timeout – a genuine deadlock (from waking the wrong condition variable, or waiting on the wrong one) will hang that thread combination until the timeout kills the whole test as a TLE.

Problem 9: The Sliding Window Maximum

Topic: Applications of Stacks/Queues – a Monotonic Deque on a Doubly Linked List **Difficulty:** 5/5 **Suggested time:** 110–130 minutes **Total hidden marks:** 52 (across 20 hidden test cases; 2 additional visible cases provided for debugging)

Story

You are given an array `arr[0..n-1]` and, for each query, a window size `k`. A “window” is any block of `k` consecutive elements `arr[i]`, `arr[i+1]`, ..., `arr[i+k-1]`; there are exactly `n-k+1` such windows (`i = 0, 1, ..., n-k`). For a given `k`, you must report the maximum element within *every* window, in order, as the window of size `k` slides one position to the right at a time across the whole array.

Example: `arr = [1, 3, -1, -3, 5, 3, 6, 7]`, `k = 3`. The windows are `[1,3,-1]`, `[3,-1,-3]`, `[-1,-3,5]`, `[-3,5,3]`, `[5,3,6]`, `[3,6,7]`, with maxima 3, 3, 5, 5, 6, 7 respectively.

Method 1: Brute force, $\Theta(n \cdot k)$

For every window, scan all `k` of its elements directly. Correct, but far too slow once both `n` and `k` get large – a single query with `n = k = 300000` would need on the order of 9×10^{10} element comparisons.

Method 2: Monotonic deque, $\Theta(n)$ per query

Maintain a deque of *indices* into `arr` with the invariant that the corresponding *values* are always in strictly decreasing order from the front of the deque to the back. Process indices `i = 0, 1, ..., n-1` in order:

1. Before adding index `i`, pop indices off the **back** of the deque as long as it is non-empty and `arr[back index] <= arr[i]` – those indices can never again be the maximum of any future window (`arr[i]` is at least as large as they are, and outlasts them in every window they’re both part of, since `i` is further right).
2. Push index `i` onto the back of the deque.
3. If the index at the **front** of the deque has fallen out of the current window (i.e. it is `<= i - k`), pop it off the front.
4. Once `i >= k - 1` (a full window ending at `i` exists), the maximum of that window is `arr[front index of deque]` – report it.

Because every index is pushed exactly once and popped at most once, from either end, across the entire run, the *total* work across all `n` steps is $\Theta(n)$ – independent of `k`.

The catch: you must build your own Deque, on a doubly linked list

You must implement both methods, and – this is the point of the exercise – the deque itself must be your **own doubly-linked-list** Deque class (`push_front`, `push_back`, `pop_front`, `pop_back`, `peek_front`, `peek_back`, each $O(1)$), not Python’s `list` or `collections.deque`.

Input format and queries

`n`

```

<n space-separated integers -- arr[0..n-1]>
q
<q lines, each one of:>
  BRUTE k
  FAST k

```

For each query, print the resulting window-maxima list, space-separated, on its own line ($1 \leq k \leq n$ is guaranteed, so this list is never empty).

Worked example

```

arr = [1, 3, -1, -3, 5, 3, 6, 7]
BRUTE 3  -> 3 3 5 5 6 7
FAST 3   -> 3 3 5 5 6 7

```

What you must implement

Complete the `Deque` class (built on a `_DNode` doubly-linked list) and the two functions `sliding_window_max_bruteforce` and `sliding_window_max_monotonic` in `student_code.py`, the latter built entirely on top of your own `Deque`. The I/O driver is complete and must not be modified.

Notes on grading

As with Problem 3, **BRUTE** is only ever asked for on inputs small enough that $\Theta(nk)$ is affordable – use it, and the provided visible cases, to sanity-check **FAST** against it yourself. Small correctness cases include the worked example above, k equal to the array length, $k = 1$, strictly increasing/decreasing arrays, repeated maximum values, all-equal arrays, negative numbers, and several k values queried on the same array in one test. Large hidden performance cases run **FAST** (never **BRUTE**) on arrays of up to 300,000 elements with a variety of k values, including a specifically adversarial “zig-zag” array (increasing then decreasing) engineered to maximize churn on the monotonic deque – an implementation that is correct but secretly $\Theta(nk)$, or one whose `Deque` isn’t genuinely $O(1)$ per operation (for instance, one built on a Python list with `pop(0)`), will not finish these within the time limit even though it passes every small case.

Problem 10: The LRU Cache

Topic: Capstone – Linked Lists + a Hash Map, worst-case $O(1)$ **Difficulty:** 5/5 (hardest in this set) **Suggested time:** 120–150 minutes **Total hidden marks:** 68 (across 20 hidden test cases; 2 additional visible cases provided for debugging)

Story

This is the hardest problem in this set, and it asks you to combine almost everything covered so far into a single data structure: a **Least-Recently-Used (LRU) cache** of fixed capacity, supporting PUT and GET, **both in worst-case $O(1)$ time** – not amortized, not “usually fast”, but $O(1)$ on *every single call*, in exactly the spirit of the Ladder List-style structures that demand worst-case bounds rather than merely amortized ones.

The cache stores up to **capacity** distinct (key, value) pairs. Every time a key is *accessed* – by a successful GET, or by a PUT that touches it (inserting it fresh, or updating its value) – that key becomes the “most recently used” (MRU) one. Whenever a PUT needs to insert a brand new key while the cache is already at capacity, the “least recently used” (LRU) key – the one that has gone the longest without being touched – is evicted first, to make room.

Commands

PUT key value	If key is already present, update its value and mark it most recently used. Otherwise insert it as most recently used; if this would exceed capacity, evict the current least-recently-used key first.
GET key	If key is present, mark it most recently used and return its value. Otherwise return NULL, with no other effect.
SIZE	Report the number of keys currently stored.
CAPACITY	Report the cache's fixed capacity.
KEYS_MRU_TO_LRU	Report every stored key, most-recently-used first, down to least-recently-used, space-separated -- or NONE if the cache is empty. (This exists purely so you and the grader can directly verify your recency ORDER is exactly right -- a subtly wrong LRU implementation very often still passes basic GET/PUT tests by coincidence while getting the eviction order wrong under more elaborate access patterns.)

Why neither a dict alone nor a linked list alone is enough

A plain dict gives $O(1)$ lookup by key, but no fast way to find “the key that was least recently touched” without scanning or maintaining a separate ordering by hand – which reduces at least one of GET/PUT to $O(n)$ or worse. A doubly linked list gives $O(1)$ insertion/removal at either end, and $O(1)$ “move this node to the front”, *once you already have a reference to the right node* – but finding *which* node corresponds to a given key, from the key alone, is $O(n)$ without something extra.

The required design

Combine the two: a **doubly linked list** of nodes ordered from most-recently-used (right after a head sentinel) to least-recently-used (right before a tail sentinel), together with a **dict mapping each key directly to its node** in that list. A lookup is an $O(1)$ dict access; once you have the node, unlinking it from wherever it sits and re-inserting it at the MRU end are both $O(1)$ linked-list operations; evicting the LRU key means reading `tail.prev` ($O(1)$), unlinking it ($O(1)$), and deleting its dict entry ($O(1)$). Every operation is therefore $O(1)$ *worst-case*, **provided** the dict genuinely stores a node-reference (rather than, say, storing keys and re-searching the linked list to find the matching node each time – which would quietly turn every operation back into $O(n)$, and is exactly the mistake several large hidden performance tests below are designed to catch). Using dummy head/tail sentinel nodes (present from the start, holding no real data, initially pointing directly at each other) removes the need to special-case an empty list or a single-node list anywhere in your code, and you are strongly encouraged to use them.

Worked example

```
capacity = 2
PUT a 1
PUT b 2
GET a      -> 1      (a is now MRU; order is a, b)
PUT c 3    (cache full; b is LRU -> evicted; order is c, a)
GET b      -> NULL   (b was evicted)
GET a      -> 1
GET c      -> 3
```

What you must implement

Complete the `LRUCache` class in `student_code.py`: `__init__`, `_remove_node`, `_insert_front`, `_touch`, `get`, `put`, `size`, and `keys_mru_to_lru`. Store the capacity as `self.capacity` (the driver reads this attribute directly for `CAPACITY`). The I/O driver at the bottom is complete and must not be modified.

Notes on grading

Small correctness cases check basic put/get, eviction of the correct (least recently used) key, that `GET` genuinely refreshes recency and can save a key from an eviction it would otherwise have suffered, that `PUT` on an *existing* key updates its value *and* refreshes its recency, capacity 1 as an edge case, and – most exactly – several `KEYS_MRU_TO_LRU` checks after specific access sequences, to directly verify your recency ordering rather than just the externally visible `GET` results. A battery of randomized stress tests (with varying small capacities) is checked against an independent `OrderedDict`-based reference model. Finally, three large hidden performance tests – one with 50,000 distinct keys, one with heavy eviction pressure under a small capacity and hundreds of thousands of operations, and one repeatedly hammering a small “hot set” of keys – run under a tight time limit that only a genuine $O(1)$ -per-operation combination of a dict and a doubly linked list will clear; any solution that resorts to scanning or re-sorting anywhere will time out here even though it may well have passed every earlier test in this problem.

Appendix: total marks summary

#	Problem	Topic	Difficulty	Hidden marks
1	The Fleet Console	OOP	Difficulty 1/5	29
2	The Data Pipeline Factory	Functional programming	Difficulty 2/5	37
3	Maximum Gain Window	Divide & conquer / asymptotics	Difficulty 3/5	44
4	The Growable Ring Buffer	Arrays / amortized analysis	Difficulty 3/5	54
5	Undo, Redo	Linked lists / stacks	Difficulty 3/5	45
6	The Interval Set	Set ADT / linked list	Difficulty 4/5	58
7	The Max Queue	Queue ADT via stacks	Difficulty 4/5	55
8	The Bounded Buffer	Concurrency	Difficulty 4/5	63
9	The Sliding Window Maximum	Stack/queue applications	Difficulty 5/5	52
10	The LRU Cache	Capstone (linked list + hash map)	Difficulty 5/5	68

Good luck. If a problem makes you question your life choices somewhere around the third hidden performance test failing with TLE, that is not a sign you are doing something wrong – it is the sign that you are doing this practice set correctly.